LINUX DEVICE DRIVER PROJECT

Linux Device Driver -Based 스마트 선풍기

Jetson Orin Nano 에서 센서 · 입력 · 표시장치 · 모터를 커널 드라이버로 연결



Team 5

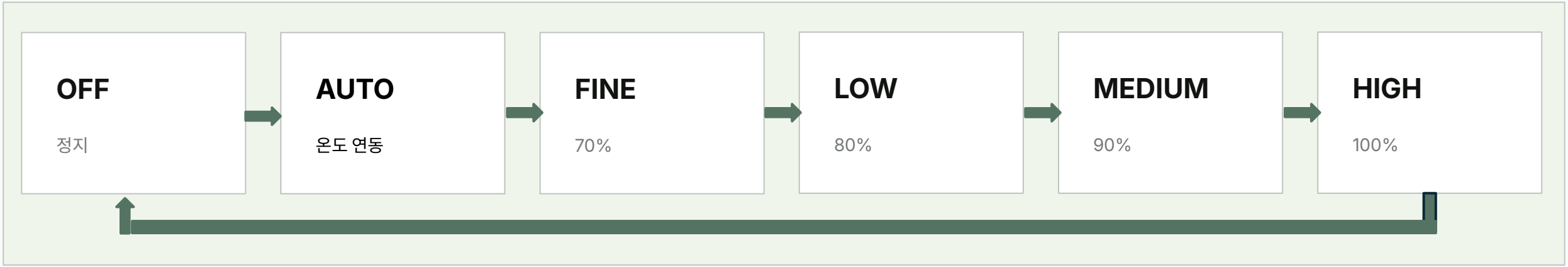
권민지 · 김승환 · 김태환 · 이세형 · 이양배

프로젝트 목표와 동작 모드

프로젝트 목표

각 장치의 커널 드라이버 구현 및 사용자 공간 애플리케이션을 통한 선풍기 통합 제어

운전 모드 버튼을 누를 때마다 OFF 부터 FINE 까지 순환



주요 기능

조작 및 설정

엔코더로 운전 모드
및 타이머 조절

자동 풍속 조절

온도에 따라
풍속 자동 조절

상태 표시

LCD·FND·LED 로
현재 상태 표시

타이머

설정 시간 종료 시
모터 정지 및 부저 알림

개발 환경

Jetson Orin Nano 에서 커널 모듈을 직접 빌드하고 적재

주요 개발 환경

개발 보드

Jetson Orin Nano

NVIDIA Orin (Tegra234)
AArch64

운영체제

Ubuntu 22.04

22.04.5 LTS
Jammy Jellyfish

커널

5.15.199-tegra

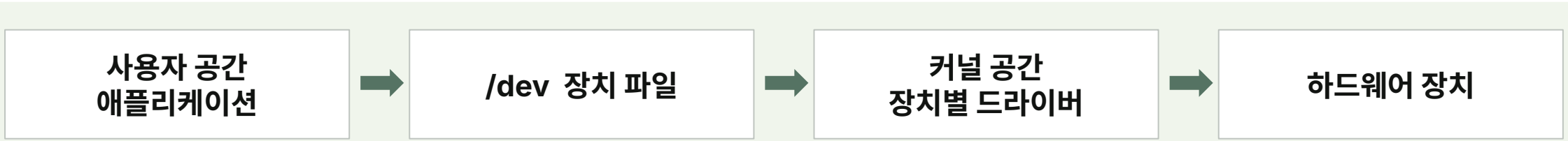
NVIDIA Tegra · PREEMPT
L4T R36.5.2

컴파일러

GCC 11.4.0

장치별 커널 모듈 빌드

장치 제어 구조



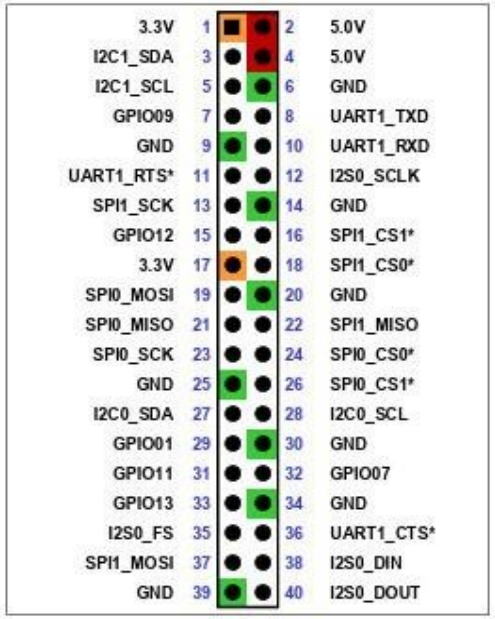
커널 모듈 (.ko) 을 적재한 뒤 , 사용자 공간 애플리케이션에서 /dev 장치 파일을 통해 제어

핀 배정과 장치 연결

장치별 통신 방식과 Jetson 40 핀 헤더 연결

장치	신호	보드 핀
LCD	I2C	27 (SDA), 28 (SCL)
로터리 엔코더	GPIO	KEY 24, S2 26, S1 22
부저	PWM	33
BMP180	I2C	3 (SDA), 5 (SCL)
LED BAR	GPIO	LED1~7 = 8, 10, 35, 38, 19, 21, 23
FND	GPIO	A~G = 7, 11, 12, 13, 15, 16, 18 / DIG3 37, DIG4 40
모터	GPIO · PWM	IN1 29, IN2 31, EnableA(PWM) 32

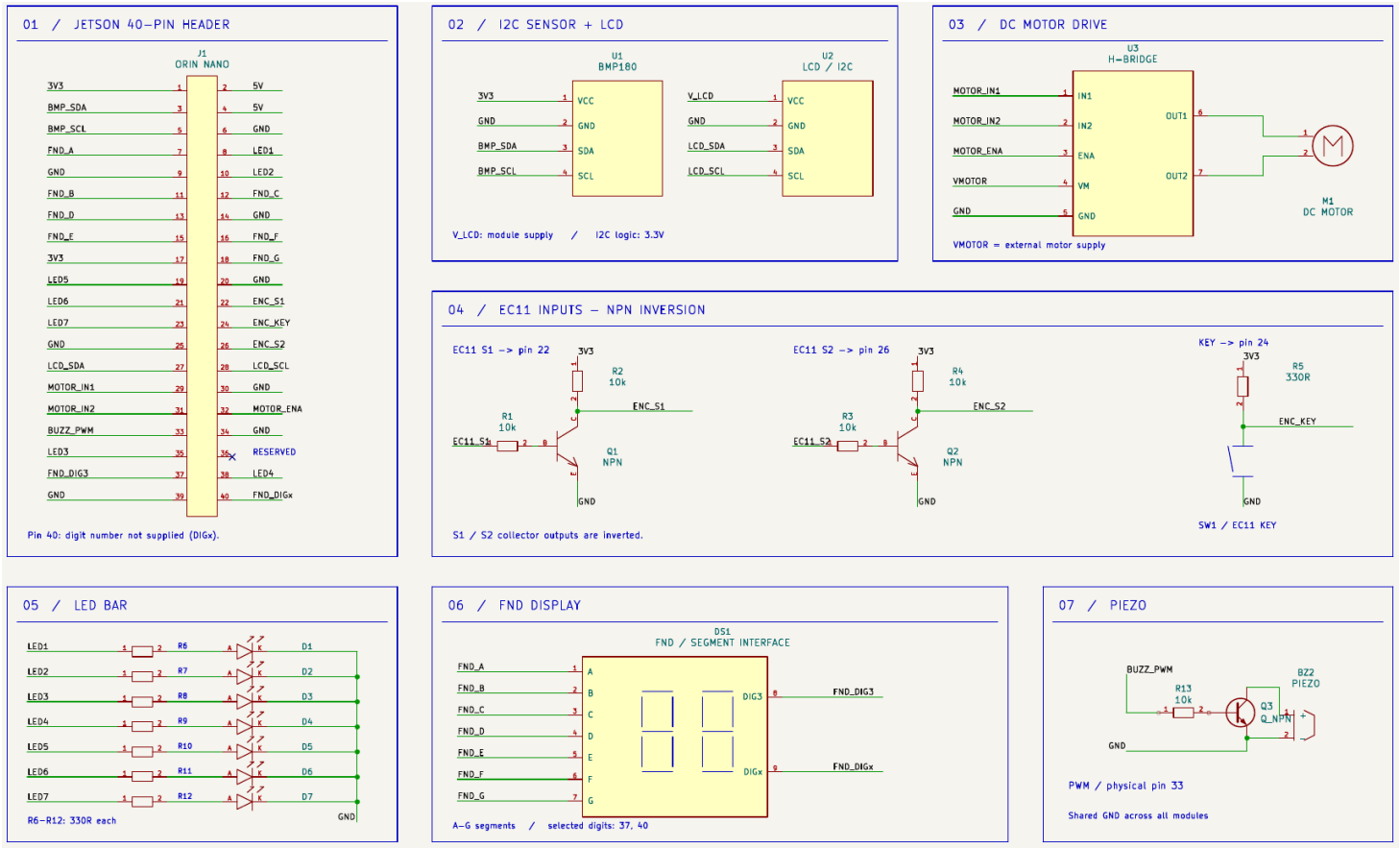
I2C 2 선 직렬 통신 · PWM 펄스 폭 조절 출력 · GPIO 범용 입출력



Jetson 40 핀 헤더 핀맵
(image/pin_map.jpg)

회로 연결도

Jetson 과 입력 · 센서 · 모터 · 표시 장치의 전체 연결 구성



연결 요약

입력 · 센서
엔코더 / BMP180

구동 · 표시
모터 / LCD / LED / FND / 부저

전원 구성
모터 외부 전원
모듈 간 공통 GND

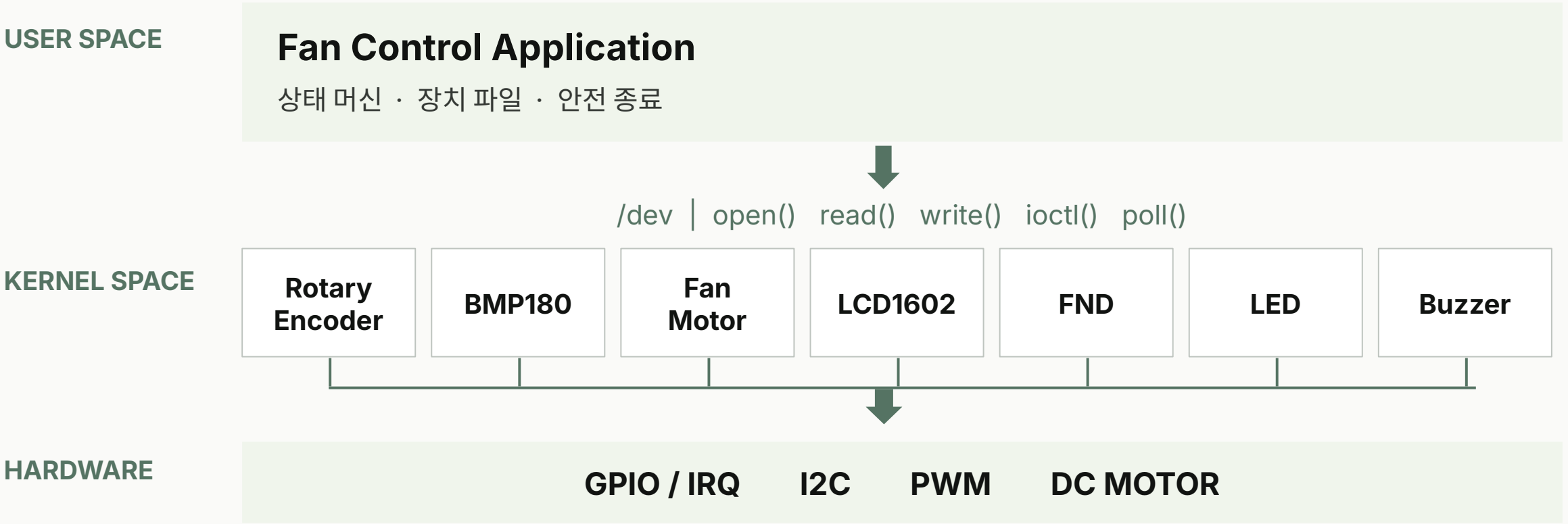
사용 부품과 데이터시트 검증

“ 부품 표기와 데이터시트를 대조해 주소 · 핀 · 타이밍을 설계 기준으로 정리 "

기능	확인 부품	근거 자료	드라이버 인터페이스
입력	EC11 계열	EC11 Forward Direction Series	GPIO · IRQ
문자 출력	QAPASS 1602A	LCD1602 호환 규격	I2C · PCF8574T
온도 측정	Bosch BMP180	Bosch 원본 데이터시트	I2C
모터 구동	L298N module	HW-095 User Guide	GPIO · PWM
시간 표시	5461AS-1 계열	NFD-5642 호환 규격	GPIO 다중화

전체 시스템 구조

사용자 앱은 운전 모드를 결정하고 , 커널 드라이버는 장치 제어와 이벤트를 처리



GPIO·IRQ·I2C·PWM 을 하나의 선풍기 제어 흐름으로 연결

드라이버 구성과 역할 분담

장치 드라이버 7 종과 제어 앱의 역할

구성 요소	장치 파일	핵심 내용	담당
BMP180	/dev/bmp180	1 초 주기 온도 · 기압 측정 , bmp180_get_data() 공개	김태환
LED BAR	/dev/led_bar	led_set_duty() 로 풍량 0·1·3·5·7 칸 표시	김태환
LCD	/dev/lcd_t	I2C(0x27) 연결 , 온도 · 기압 1 초 갱신 표시	이양배
로터리 엔코더	/dev/lt_en	KEY 눌림 read·poll, CW·CCW 회전 kfifo 적재	이양배
FND 타이머	/dev/fnd	00~99 분 타이머 , 회전 ±1 분 , ioctl 6 종	이세형
모터	/dev/fan_motor	write(mode) 로 6 모드 전환 , AUTO 는 온도 연동	이세형
부저	/dev/buzz_t	write(1) 로 PWM 4 음 알림 , buzzer_fnd_complete() 공개	권민지
Application	fan_app.c	KEY poll· 모드 순환 , 타이머 만료 시 모터 OFF· 부저	권민지

데이터시트 기반 드라이버 설계

데이터시트의 수치와 파형을 커널 API· 상태 처리 · 오류 조건으로 반영

설계 과정

01 부품 식별	02 제약 추출	03 드라이버 설계	04 통합 검증
모델 표기와 인터페이스 확인	주소 , 핀맵 , 레지스터 , 타이밍	상태 , 파일 연산 , 동시성	로그 , 장치 파일 , 실제 동작

드라이버 반영 예시

BMP180 0x77	LCD Enable pulse	EC11 A/B 위상	L298N ENA
I2C client 통신	마이크로초 지연	IRQ 상태 전이	PWM duty 제어

데이터시트 반영 요약

데이터시트 요구값이 실제 코드에 반영된 위치

부품	핵심 요구값	드라이버 반영	근거 쪽
EC11	15 pulse/360°, 채터링 ⩽ 3ms, 푸시 바운스 ⩽ 10ms	lt_en.c 전이 테이블 · 20ms 디바운스	1/5 ~ 5/5
LCD1602	초기화 시퀀스 , 명령 실행 40µs · 1.64ms	lcd_t.c 초기화 지연 · usleep	p.12, 13
BMP180	주소 0x77, 보정계수 0xAA–0xBF, 변환 4.5 · 7.5ms	bmp180.c probe · 변환 대기	p.11, 13, 18, 20, 21
L298N	전원 5–35V, 피크 2A, 제어 High ≥ 2.3V	fan_motor.c IN1 · IN2 · ENA PWM	p.1, 3, 4
7 세그먼트	4 자리 12 핀 , 세그먼트 If 20mA 기준	fnd_driver.c 다중화 · 패턴 테이블	p.1, 2

EC11 로터리 엔코더 입력 드라이버

A/B 2 상 신호의 상태 전이로 회전 방향을 판별하고 버튼 입력을 전달



S1/S2 2 상 출력 , 15° / 20 Steps

20ms
KEY SOFTWARE DEBOUNCE

16
LOOKUP TABLE

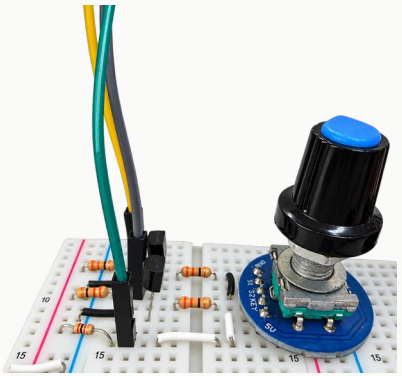
±4
CW / CCW DECISION

데이터시트 요구사항

- 회전 접점 채터링 최대 3ms
- 푸시 스위치 bounce 최대 10ms
- A 상과 B 상의 순서로 방향 판별

드라이버 구현

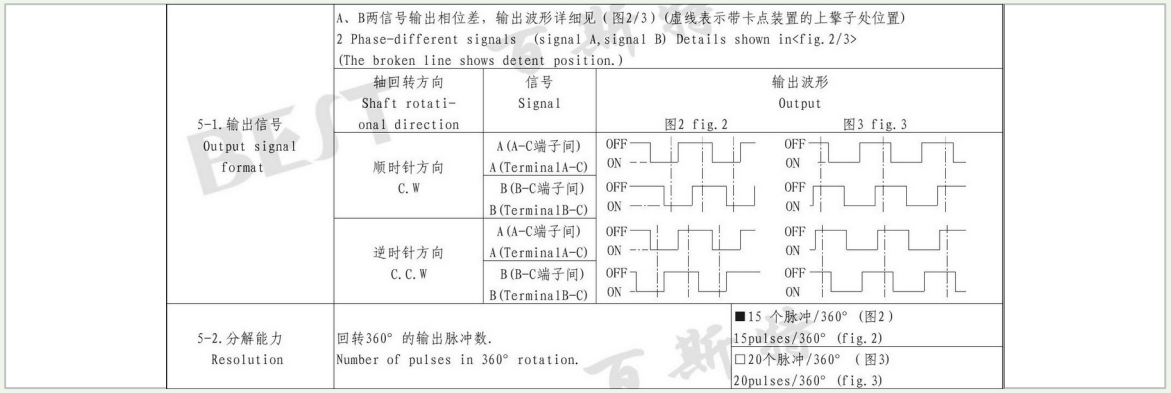
```
static const int rotary_table[16] = {
    /*
    cw  0001 1  0111 7  1110 14 1000 8
    ccw 0010 2  1011 11 1101 13 0100 4
    */
     0,  1, -1,  0,
    -1,  0,  0,  1,
     1,  0,  0, -1,
     0, -1,  1,  0
};
```



```
IRQ       S1/S2 상승·하강 에지 등록
STATE     이전 / 현재 2 비트 조합을 전이 테이블로 누적
EVENT     KEY 는 wait queue·poll(), 회전은 kfifo → FND 인출
```

EC11 데이터시트 발췌

출력 파형 · 분해능 · 채터링 조건을 인터럽트와 상태 전이 처리에 반영



EC11 Forward Direction Series Specification 2/5 5-1 Output signal, 5-2 Resolution

20
Steps 15 / 360°

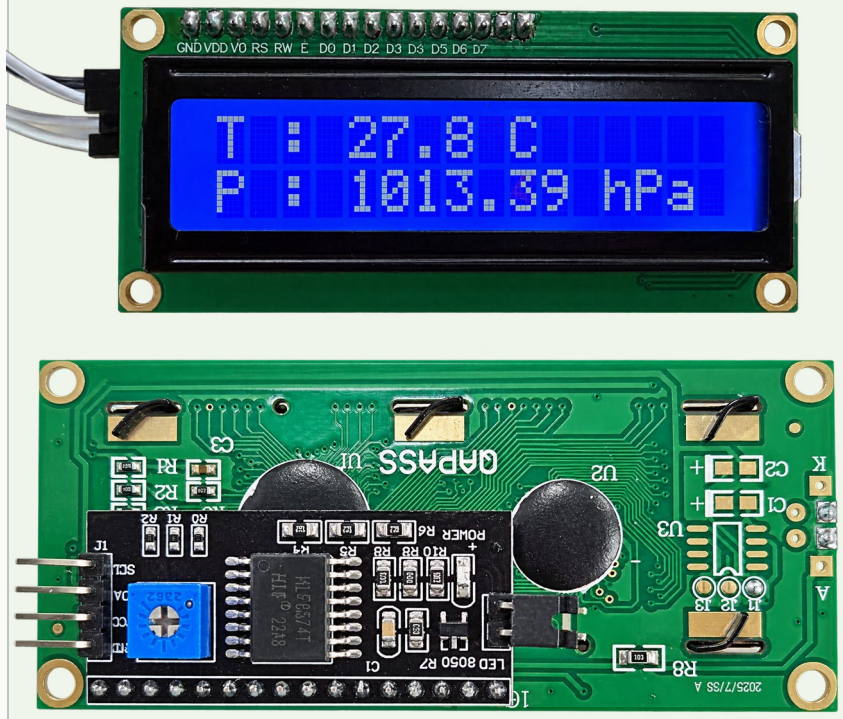
≤3ms
CHATTERING t1 · t3

≤10ms
PUSH SWITCH BOUNCE

항목	데이터시트	드라이버 구현
출력 신호	A · B 2 상 위상차 출력 , CW · CCW 파형 구분	S1 · S2 양 에지 인터럽트 , 2 비트 현재 상태 읽기
방향 판정	A 상 기준 소프트웨어 설계 권장 (4-3)	이전 · 현재 4 비트 인덱스로 rotary_table[16] 조회
분해능	20 Steps / 360° (Fig.2)	전이 합계 ±4 도달 시 CW(1) · CCW(2) 이벤트 1 회
채터링	t1 · t3 ≤ 3ms, 바운스 t2 ≤ 2ms	불가능한 전이는 테이블 값 0, 카운트 제외
푸시 스위치	바운스 10ms 이하	KEY 하강 에지 20ms 디바운스 , wait queue 깨움

LCD1602 I2C 출력 드라이버

I2C 기반 LCD 제어 및 온도 · 기압 정보 표시



QAPASS 1602A + PCF8574T backpack

PCF8574 비트 배치

RS 0x01 CMD/CHAR	**RW** 0x02 READ/WRITE	**EN** 0x04 DATA_CONTROL	**BL** 0x08 BACK_LIGHT

4-bit 초기화 시퀀스

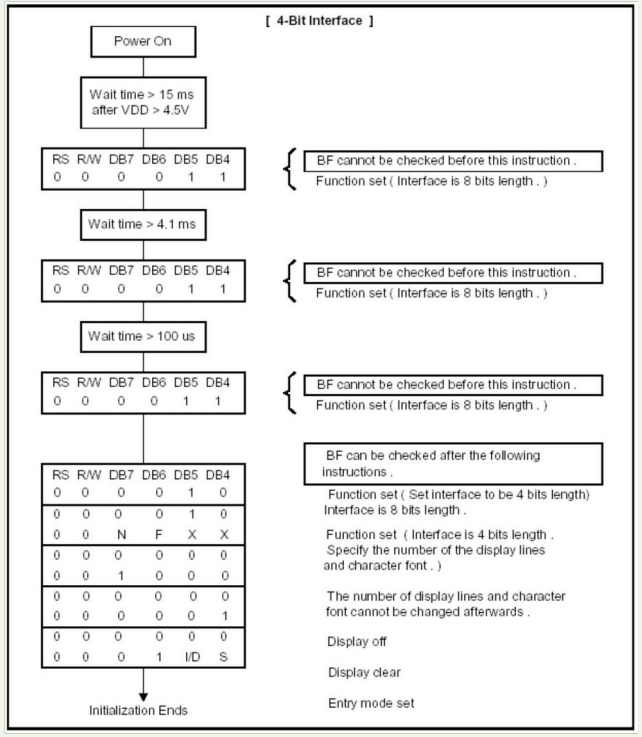
50ms	0x30	5ms	0x30	5ms	0x30	0x20

설정 명령 0x28 · 0x0C · 0x06 · 0x01

/dev/lcd_t	실행 시간	상태 표시
최대 32 바이트 write 한 줄 문자 출력	Clear/Home 은 2ms 이상 대기 일반 명령은 마이크로초 지연	BMP180 최신 온도 · 기압 LCD 두 줄에 1 초마다 표시

LCD1602 데이터시트 발췌

초기화 순서와 명령별 실행 시간을 지연 함수에 반영



Waveshare LCD1602 Specification
§12 Initialization Sequence (p.12)

>15ms
POWER-ON WAIT

1.64ms
CLEAR · HOME E-CYCLE

40µs
GENERAL INSTRUCTION

항목	데이터시트	드라이버 구현
전원 인가	VDD 4.5V 이후 15ms 초과 대기	msleep(50) 후 초기화 시작
8 비트 지정	0x30 세 번 , 4.1ms·100µs 초과 대기	0x30 → 5ms → 0x30 → 5ms → 0x30 → 200µs
4 비트 전환	Function set 0x20 (DB5=1)	0x20 전송 뒤 4 비트 명령 모드 진입
설정 명령	Function set · Display · Clear · Entry mode	0x28 · 0x0C · 0x06 · 0x01 순서로 전송
실행 시간	Clear·Home 1.64ms, 그 외 40µs	Clear·Home 2ms 이상 , 니블마다 50µs
신호 연결	RS · R/W · E · DB4~7 (4 비트 인터페이스)	PCF8574 비트 RS 0x01 · RW 0x02 · EN 0x04 · BL 0x08

데이터시트 예시는 Display off 후 Clear 순서 , 구현은 Display on(0x0C) 을 포함한 순서

BMP180 센서 드라이버

I2C 로 온도와 기압을 측정하고 보정된 최신값을 다른 모듈에 제공



BMP180 breakout module · I2C interface

0x77
I2C ADDRESS

11
CALIBRATION COEFFICIENTS

0x55
CHIP ID

데이터시트 해석

- 보정계수 레지스터 0xAA–0xBF
- 온도 명령 0x2E, 최대 변환 시간 4.5ms
- OSS=1 압력 측정 , 최대 변환 시간 7.5ms

드라이버 구현

```
probe   chip ID 검증 , 보정계수 22 바이트 로드
work    1 초 주기로 온도와 기압 갱신
lock    mutex 로 I2C 보호 , spinlock 으로 최신값 공유
```

BMP180 데이터시트 발췌

보정계수와 측정 순서를 검증하고 변환 시간 이후 데이터를 읽음

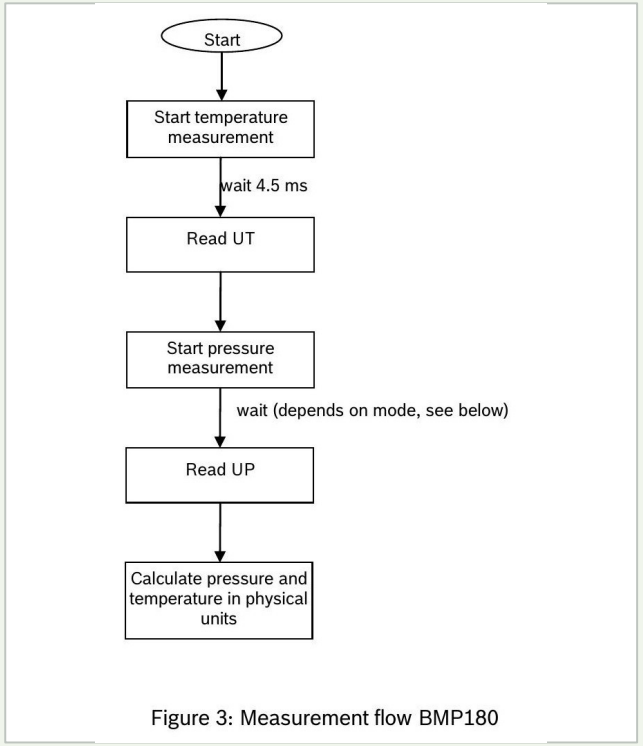


Figure 3: Measurement flow BMP180

Bosch BMP180 Data sheet Rev 2.5
Figure 3 Measurement flow (p.11)

4.5ms
TEMPERATURE CONVERSION

7.5ms
PRESSURE CONV. (OSS=1)

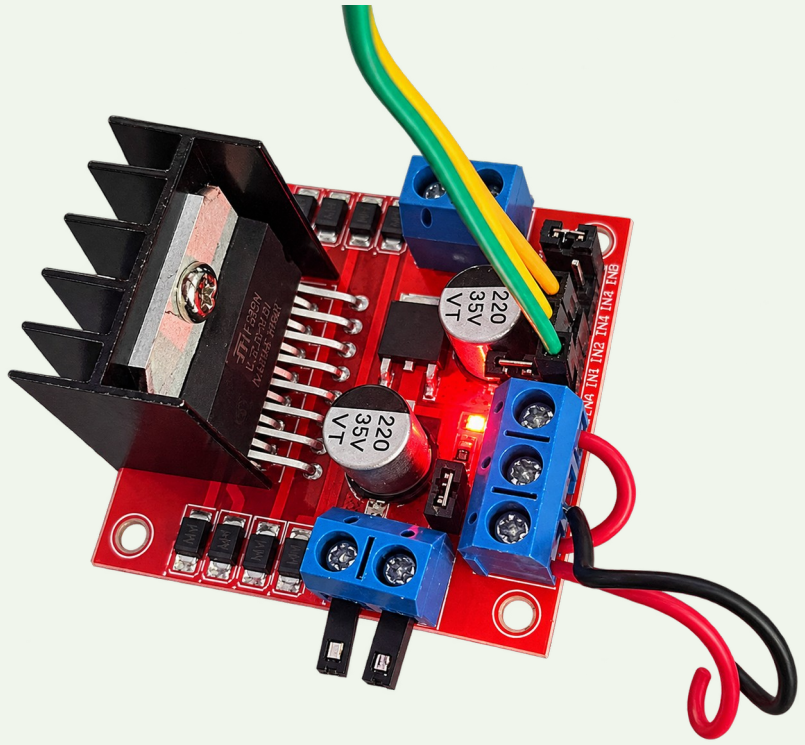
176bit
CALIBRATION E²PROM

항목	데이터시트	드라이버 구현
측정 순서	온도 → 4.5ms 대기 → 압력 → 계산 (Fig.3)	온도 · 압력 순차 측정 후 보정식 계산
I2C 주소	7 비트 0x77 (0xEE 쓰기 / 0xEF 읽기)	I2C 클라이언트 주소 0x77 등록
칩 확인	0xD0 레지스터 값 0x55	probe 단계에서 칩 ID 검증
보정계수	0xAA–0xBF 11 워드 , 0 또는 0xFFFF 는 오류	22 바이트 로드 후 값 검사
변환 명령	0xF4 에 0x2E (온도) 0x34+(OSS<<6) (압력)	OSS=1, 변환 후 5~6ms·8~9ms 대기
샘플링	표준 모드 최대 128 회 /s 온도는 1 초 1 회로 충분	1 초 주기 delayed work 로 갱신

구현 메모 I2C 접근은 mutex, 최신값 공유는 spinlock 으로 분리

L298N 모터 드라이버

GPIO 로 방향을 설정하고 ENA 의 PWM duty 로 풍속을 제어



5–35V board supply · 2A peak · H-bridge

20kHz
PWM FREQUENCY

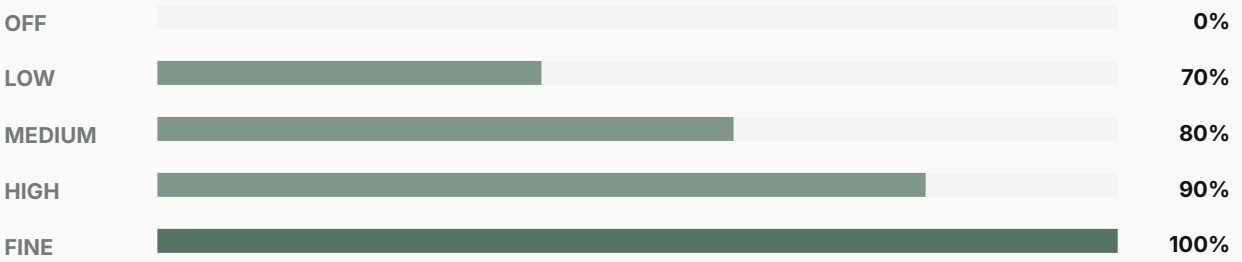
3
PWM ID

453 / 454
IN1 / IN2 GPIO

방향 제어

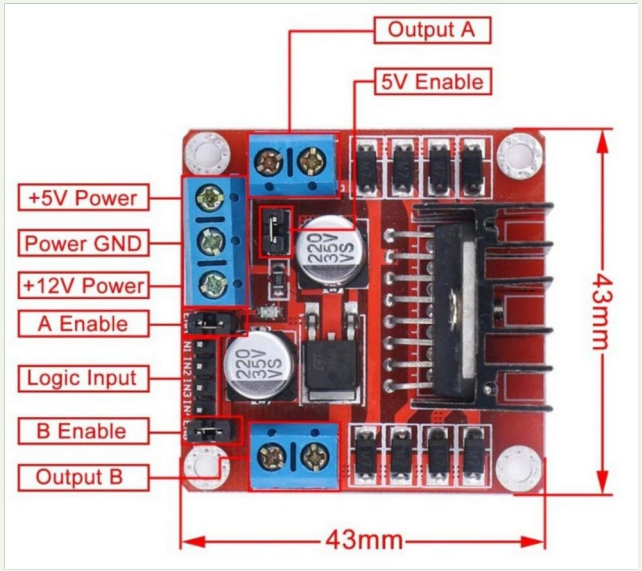
정방향	IN1 HIGH	IN2 LOW
정지	IN1 LOW	IN2 LOW

운전 모드별 duty



L298N 데이터시트 발췌

제어 입력 조건과 ENA 신호를 GPIO·PWM 구동에 반영



L298N Dual H-Bridge Motor Driver
User Guide p.3 Board Pins Function

5–35V
POWER SUPPLY

2A
PEAK CURRENT

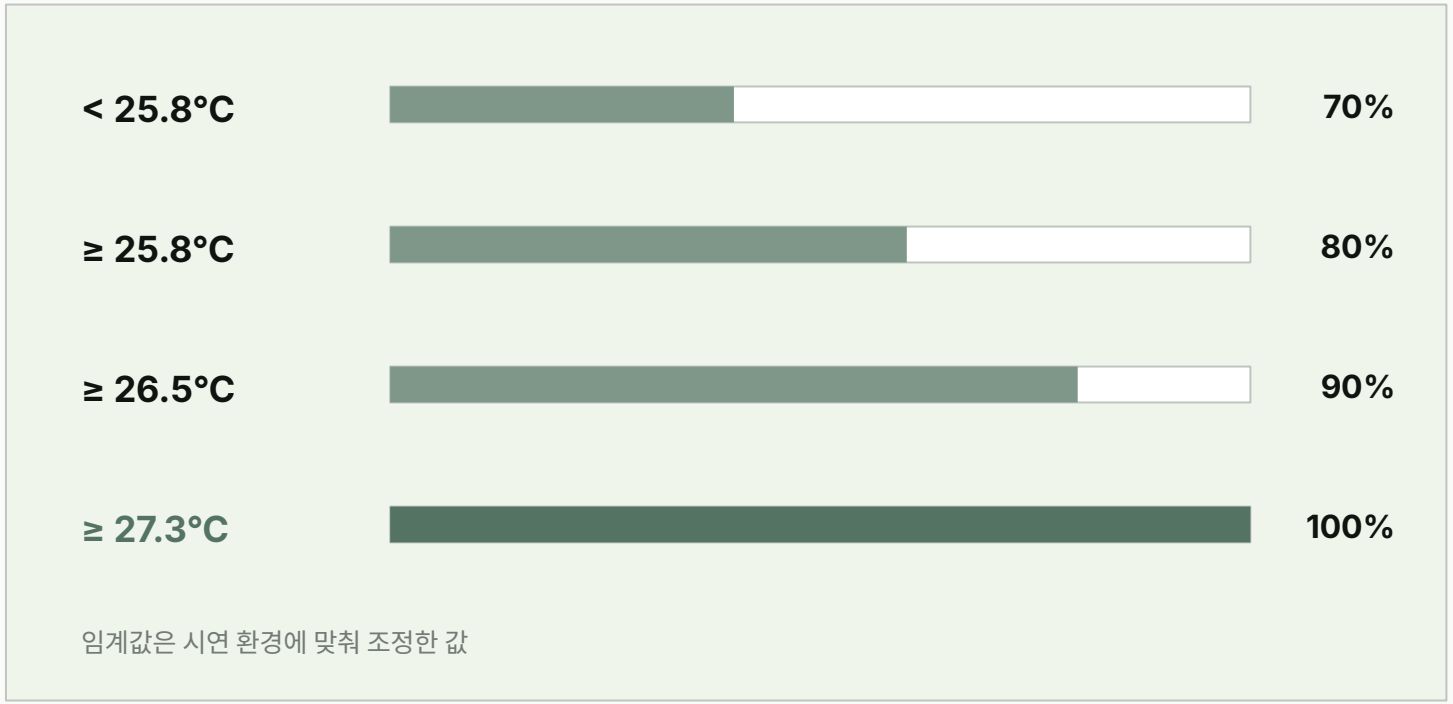
≥2.3V
LOGIC HIGH INPUT

항목	데이터시트	드라이버 구현
제어 신호	Low ≤ 1.5V, High ≥ 2.3V	GPIO 출력으로 IN1 · IN2 직접 구동
방향 제어	IN1 HIGH · IN2 LOW 시 정방향 (예제)	동작 시 IN1 HIGH, IN2 LOW 설정
정지	IN1 · IN2 동일 레벨 (예제는 둘 다 HIGH)	OFF 시 IN1 · IN2 둘 다 LOW
속도 제어	Enable High 에서 제어 신호 유효	ENA 에 PWM 입력 , 주기 50µs(20kHz)
채널 구성	Logic Input IN1~IN4, Output A · B	모터 A 채널만 사용 (IN1 29, IN2 31, ENA 32)
구동 전류	피크 2A, 최대 전력 20W (75°C)	모터 전류 정량 측정은 개선 과제

구현 메모	PWM ID 3, GPIO 453 · 454 는 모듈 파라미터로 변경 가능

온도 기반 자동 풍속 제어

AUTO 모드에서 1초마다 최신 온도를 확인해 구간별 PWM duty를 선택



임계값은 시연 환경에 맞춰 조정한 값

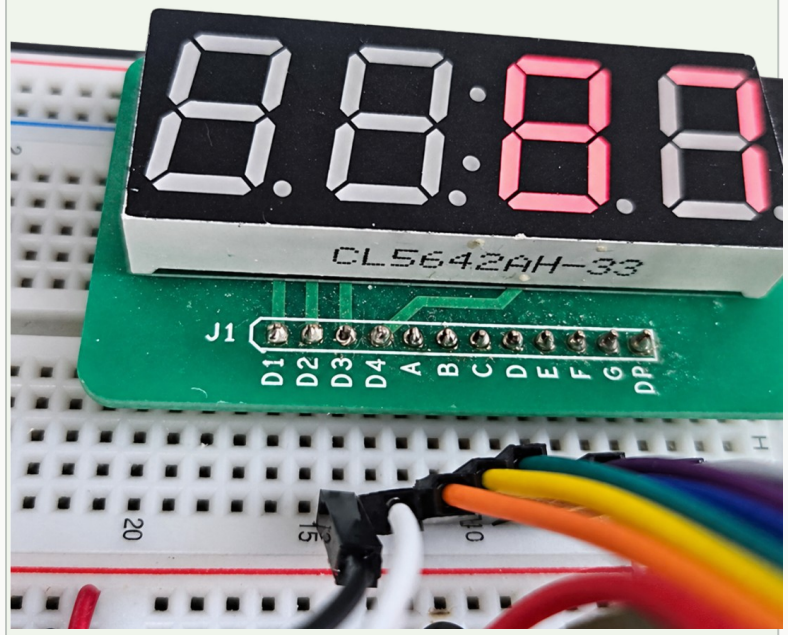
제어 설계

01	**캐시값 사용**	I2C 경합과 응답 지연 감소
02	**주기적 갱신**	delayed work를 1초마다 실행
03	**fail-safe**	센서 오류 시 모터 정지

개선안 경계 온도에 hysteresis 적용

4 자리 7 세그먼트 타이머

7 세그먼트 두 자리 분 단위 타이머 구현



실물 표기 5461AS-1 · 호환 자료 NFD-5642 계열

4ms
DIGIT REFRESH

2
ACTIVE DIGITS

00–99
TIMER MINUTES

다중화 방식

DIGIT 1	DIGIT 2	DIGIT 1	DIGIT 2	DIGIT 1	DIGIT 2

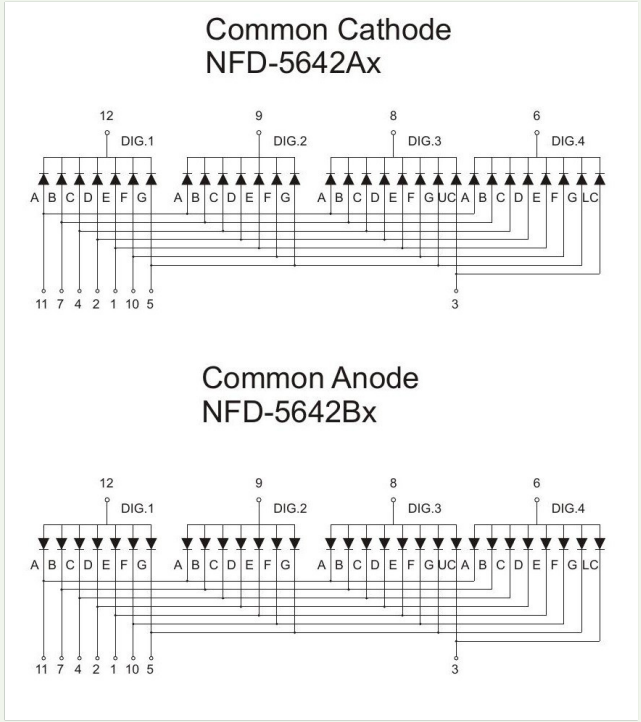
두 자리를 4ms 간격으로 번갈아 켜서 동시에 보이도록 구성

사용자 인터페이스

`write`	타이머 분 입력
`ioctl`	START · STOP · RESET · GET_REMAINING · DISPLAY ON/OFF
`read`	만료 여부 반환 (0/1)
`encoder`	CW·CCW 이벤트로 ±1 분 조절 (20ms 주기 인출)

7 세그먼트 데이터시트 발췌

7 세그먼트 핀맵 및 공통 캐소드 구조 확인



NFD-5642 Series 4-Digit Display
Page 2 Internal Circuit Diagram

4
DIGITS (DIG.1 ~ DIG.4)

12
TOTAL PINS

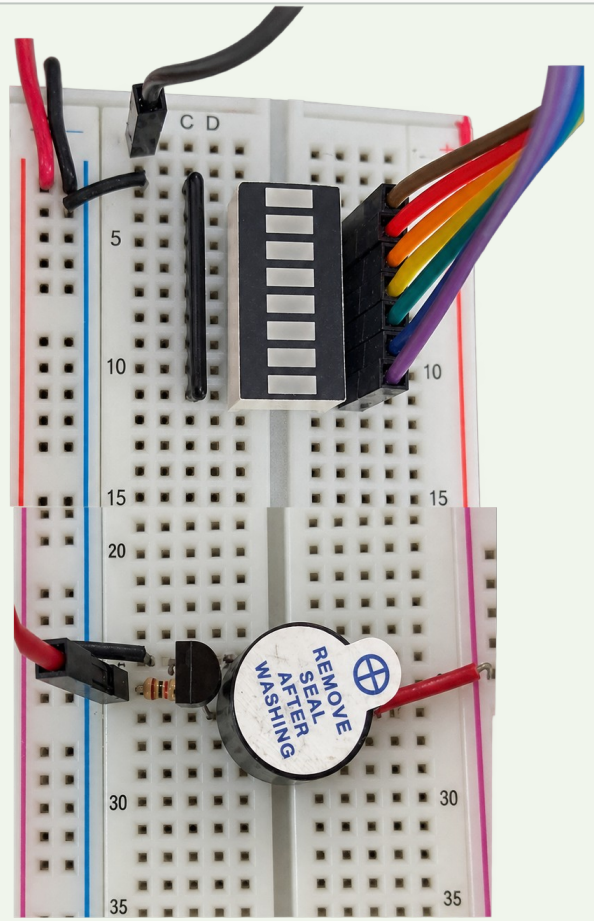
20mA
TEST CURRENT / SEGMENT

항목	데이터시트	드라이버 구현
구조	NFD-5642Ax 공통 캐소드 · Bx 공통 애노드	공통 캐소드 기준 , DIG 핀 LOW 활성
세그먼트 핀	A~G = 11 · 7 · 4 · 2 · 1 · 10 · 5	GPIO 7 개를 A~G 에 연결 (보드 7·11·12·13·15·16·18)
자릿수 핀	DIG.1~4 = 12 · 9 · 8 · 6	DIG3 · DIG4 (보드 37 · 40) 두 자리 사용
숫자 패턴	a~g 세그먼트 점등 조합	digit_pattern[10] 비트마스크 (0x3F ~ 0x6F)
구동 전류	세그먼트당 If 20mA 기준 (Ta 25°C)	4ms 간격 두 자리 교대 시분할 구동
모델 차이	NFD-5642 시리즈 자료	실물 5461AS-1 계열 , 핀맵 실물 확인

호환 자료 주의 실물 표기와 데이터시트 모델이 달라 핀맵과 공통 방식은 실물에서 재확인

LED bar 와 부저 출력

풍속은 LED 점등 개수로 표시하고 , 타이머 만료 시 PWM 알림음을 재생



실물 10-segment LED bar · 현재 드라이버는 7 GPIO 사용

풍속 표시

70%	■ □ □ □ □ □ □
80%	■ ■ ■ □ □ □ □
90%	■ ■ ■ ■ ■ □ □
100%	■ ■ ■ ■ ■ ■ ■

80ms delayed work 애니메이션 · 점등 수는 led_b.c 매핑 기준

부저 알림 패턴

2093Hz	2637Hz	2093Hz	2637Hz
120ms	120ms	120ms	350ms

PWM 50% duty · workqueue 에서 음 순서 실행

사용자 애플리케이션과 상태 전이

엔코더 버튼으로 모드를 바꾸고 타이머 만료 시 모터를 정지

OFF → AUTO → FINE → LOW → MEDIUM → HIGH

HIGH 다음 OFF 로 순환

fan_app.c 통합 경로

입력 감지	poll(encoder_fd, 200ms)
모드 전달	read(KEY) → next_mode() → write(motor_fd)
표시 제어	AUTO 는 FND 표시 OFF / 나머지는 ON / OFF 는 타이머 STOP
타이머 만료	모터 OFF → FND RESET → write(buzz_fd, 1)

안전 종료

01	**Signal 처리**	SIGINT 와 SIGTERM 감지	02	**모터 OFF**	종료 전 정지 명령 전달
03	**자원 정리**	장치 close, FND 표시 OFF			

드라이버 간 연동과 동시성

인터럽트는 이벤트를 기록하고 지연 · 반복 작업은 workqueue 로 분리

이벤트 입력

IRQ

S1/S2 edge

kfifo

CW·CCW 회전 이벤트 · FND 20ms 인출

wait queue

poll 대기 해제

주기 작업

delayed work

센서 1 초 갱신

delayed work

FND 4ms refresh

workqueue

부저 음 순서

공유 상태

mutex

I2C 와 모터 상태

spinlock

최신 센서값

atomic

이벤트 수와 stop

모듈 제거 시 delayed work 와 PWM 을 먼저 중단해 해제된 자원 접근을 방지

모듈 연동과 적재 순서

공유 심볼로 센서 · 모터 · 표시장치를 연결하고 제공 모듈부터 적재

공개 함수	제공 → 사용	용도	상태
bmp180_get_data()	bmp180 → fan_motor, lcd_t	온도 · 기압 최신값 전달	**데이터 연동**
led_set_duty()	led_b → fan_motor	풍량 LED 칸수 갱신	**모터 제어와 연동**
_cw_ccw_event()	lt_en → fnd_driver	회전 이벤트 인출 (CW 1 / CCW 2)	**FND 에 입력 값 연동**
buzzer_fnd_complete()	buzz_t → app (write)	타이머 만료 알림음	**앱 호출하여 동작**

모듈 적재 순서

bmp180 → **led_b_dev** → **led_b** → **buzz_t** → **lt_en** → **fan_motor** → **fnd_driver** → **lcd_t**

적재 규칙 호출되는 모듈부터 적재한다 . 순서가 바뀌면 Unknown symbol 오류가 나며 , 제거는 역순으로 진행한다 .

통합 문제와 해결

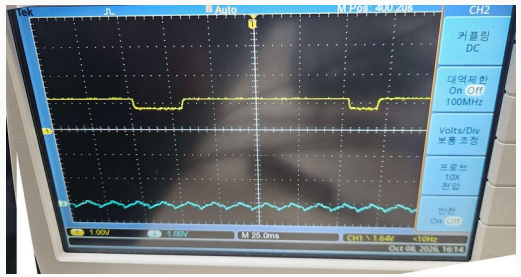
핀 · 전원 · 타이밍과 모듈 간 데이터 규약을 맞춰 통합

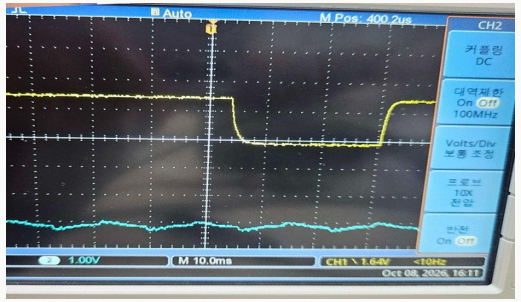
통합 과정의 문제와 해결

01

엔코더 입력 이상

입력 전압 · 방향 값 점검
NPN 반전 회로 추가





02

LCD·I2C 영향

외부 전원 사용
공통 GND 연결 확인





```
[14296.994448] failed!!!!!!!!!!!!!!
[14296.996835] failed!!!!!!!!!!!!!!
[14297.002678] failed!!!!!!!!!!!!!!
[14297.011430] failed!!!!!!!!!!!!!!
[14297.016004] failed!!!!!!!!!!!!!!
[14297.017493] failed!!!!!!!!!!!!!!
```

03

통합 실행 간 타이머 미동작

타이머 종료 조건문 논리 점검
속도 정의 재구성

```
[ 1000.449386] fnd: encoder CW, remaining=60 sec
[ 1000.553161] p_pa : 101139, temp : 272
[ 1000.985386] temp = 272, pressure = 101139
[ 1001.577164] p_pa : 101141, temp : 272
[ 1002.041352] temp = 272, pressure = 101141
[ 1002.601188] p_pa : 101141, temp : 272
[ 1003.097347] temp = 272, pressure = 101141
[ 1059.943722] p_pa : 101144, temp : 273
[ 1060.119885] temp = 273, pressure = 101144
[ 1060.967313] p_pa : 101142, temp : 272
[ 1061.175834] temp = 272, pressure = 101142
[ 1061.991645] p_pa : 101147, temp : 272
```

```
if (mode >= FAN_FINE && mode <= FAN_HIGH) {
        __u32 timer_signal;
        __u32 remaining_sec;
        ssize_t received;
```

04

공유 심볼 정의 Value 불일치

통합 코드 매핑 수정
70~100% 를 1·3·5·7 단으로

```
#define LED_DUTY_OFF      0
//#define LED_DUTY_BREEZE  60      /* 미풍 */
#define LED_DUTY_BREEZE   70      /* 미풍 */
#define LED_DUTY_WEAK     80      /* 약풍 */
#define LED_DUTY_MID      90      /* 중풍 */
#define LED_DUTY_STRONG   100     /* 강풍 */
```

```
static int duty_to_count(unsigned int duty)
{
    switch (duty) {
    case LED_DUTY_OFF:    return 0;
    case LED_DUTY_BREEZE: return 1;
    case LED_DUTY_WEAK:   return 3;
    case LED_DUTY_MID:    return 5;
    case LED_DUTY_STRONG: return 7;
    default:              return -EINVAL;   /* 약속된 5개 값 외에는 거부 (LED는 그대로) */
    }
}
```

구현 현황

6 / 6
IMPLEMENTED (SOURCE)

0
IN PROGRESS

0
NOT IMPLEMENTED

연동 항목	상태	확인 근거
엔코더 버튼 → 앱 → 모터 모드 순환	구현	poll·read·next_mode·write
모터 → LED BAR 연동	구현	LED 는 70·80·90·100% → 1·3·5·7 칸 매핑
BMP180 → 모터 (AUTO) · LCD 표시	구현	bmp180_get_data() 공유
타이머 만료 → 모터 OFF	구현 (fan_app.c)	만료 시 모터 OFF · FND RESET · 부저 write
엔코더 회전 → FND 시간 조절	구현	FND 가 20ms 마다 _cw_ccw_event() 인출 , ±1 분 조절
FND 만료 → 부저 알림	구현 (앱 호출)	앱이 /dev/buzz_t 에 write → buzzer_fnd_complete(1)

시연 영상

입력부터 모터 ·LED· 화면 · 부저까지 하나의 선풍기 시스템으로 동작

시연에서 확인할 기능

01 **엔코더 조작**

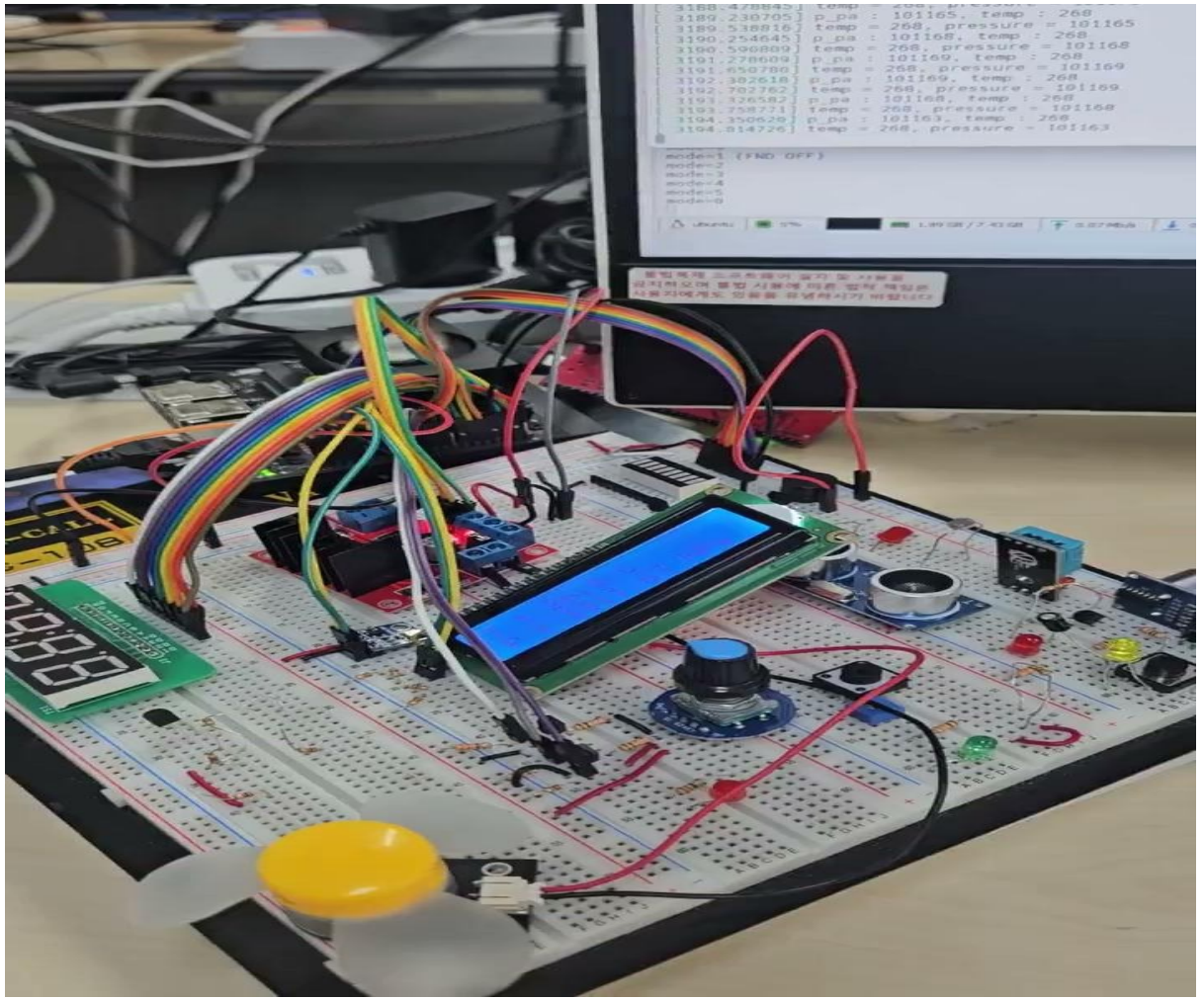
운전 모드 전환과 타이머 설정

02 **풍속 · 상태 표시**

모터 ·LED BAR·LCD·FND 반응

03 **자동 제어 · 알림**

온도 연동과 타이머 만료 처리



업무 분장

팀원별 담당 드라이버와 통합 역할

권

권민지

Application

Buzzer Driver

장치 파일

fan_app.c
/dev/buzz_t

김

김승환

Button Input

Fan Motor Control

장치 파일

/dev/lt_en
/dev/fan_motor

김

김태환

LED Driver

기압 · 온도 (BMP180) Driver

장치 파일

/dev/led_bar
/dev/bmp180

이

이세형

Motor Driver

Timer Driver (FND)

장치 파일

/dev/fan_motor
/dev/fnd

이

이양배

LCD Driver

Rotary Encoder Driver

장치 파일

/dev/lcd_t
/dev/lt_en

프로젝트 소감

팀원별 느낀 점과 어려움 · 극복 과정 , 마무리 소회

김승환

느낀 점

중도 참여했지만 , 팀원들과 진행 상황을 공유하며 프로젝트에 잘 적응할 수 있었다 .

어려움과 극복

기존 코드와 진행 상황을 파악하는 데 어려움이 있었지만 , 팀원들의 도움과 소통으로 극복했다 .

마무리 소회

협업과 소통의 중요성을 느끼고 , 함께 프로젝트를 완성해 뿟깊었다 .

김태환

느낀 점

드라이버 하나만 잘 만드는 것보다 , 다른 모듈과 값과 순서를 맞춰 연동하는 것이 더 중요하다는 것을 배웠다 .

어려움과 극복

LCD 와 모터가 함께 읽
어야 했는데 , 읽는 함수를 EXPORT_S
YMBOL 로 내보내고 공용 헤더에 선언
해서 해결하였다 .

마무리 소회

각자 만든 드라이버가 하나로 합쳐져 팬과 LED, 화면이 함께 반응하는 모습을 보며 보람을 느꼈다 .

이세형

느낀 점

이론으로만 배웠던 내용을 Jetson 보드를 통해 직접 구현하면서 다시 복습할 수 있어 즐거웠다 .

어려움과 극복

모터 드라이버와 LED 바 드라이버를 연동하는 과정에서 Duty 값에 비례하여 LED 바가 점등되지 않는 버그가 발생하였지만 , 코드를 차근차근 분석하여 문제를 해결하였다 .

마무리 소회

이번 프로젝트를 통해 임베디드 시스템 개발의 재미와 성취감을 느낄 수 있었다 .

이양배

느낀 점

LCD 와 로터리 엔코더 드라이버를 직접 구현하고 다른 팀원의 모터 · 센서 드라이버와 통합하면서 , 개별 기능의 완성도만큼 모듈 간 인터페이스와 동작 순서를 맞추는 일이 중요하다는 것을 느꼈다 .

어려움과 극복

엔코더 연결 시 전압과 방향 값이 비정상적으로 출력되고 , 모터가 LCD
·I2C 동작에 영향을 주는 문제가 있었다 . 외부 전원을 이용하여 해결하였다 .

마무리 소회

타이머와 LCD 표시까지 하나의 선풍기 시스템으로 합치는 과정에서 여러 오류를 만났지만 , 이슈를 기록하고 원인을 하나씩 검증하며 완성할 수 있었다 . 드라이버 간 연동과 협업의 중요성을 배운 프로젝트였다 .

권민지

느낀 점

그냥 이론으로 배우던 것을 실제 구현하고 하드웨어까지 작동하여 적용할 수 있어서 의미 있었다 .

어려움과 극복

전원 공급에도 부저가 정상 작동하지 않아 , 트랜지스터를 이용한 PWM 구동으로 해결하였다 .

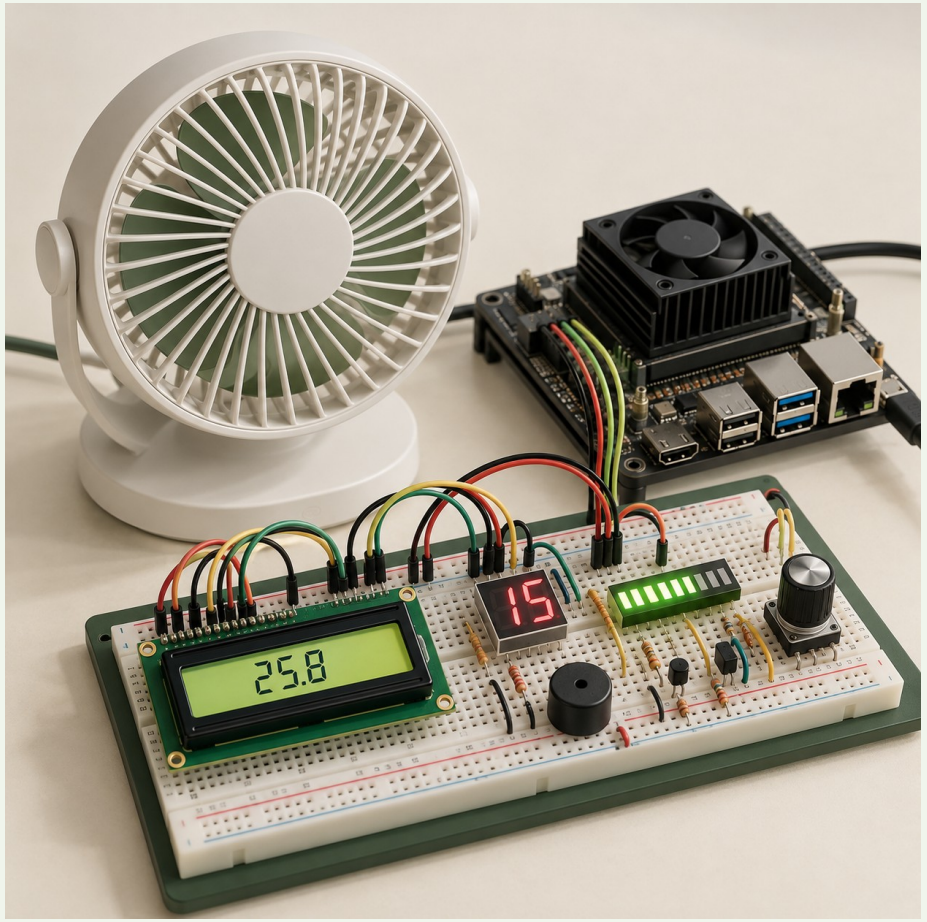
마무리 소회

구동 문제를 해결하며 하드웨어와 소프트웨어를 함께 이해하는 중요성을 배웠다 . 팀원들과 장치를 연동해 선풍기를 완성하며 협업의 보람을 느꼈다 .

QUESTIONS & ANSWERS

Q&A

질문 받겠습니다
Linux Device Driver 스마트 선풍기



CODE LINK